Design and Implementation of an Instruction and Data Memory Hierarchy Using Harvard Architecture

Pratiti Paul, Anuradha Raghuwanshi, Ganga Raghuwanshi, and Alisha Gupta

MCA Computer Architecture Coursework

Abstract

This paper presents the design and implementation of Vector Cores, a simplified 16-bit, single-cycle processor based on Harvard Architecture using Verilog HDL. The processor uses separate instruction and data memories, a 4-bit Program Counter, a Control Unit, a four-entry 8-bit Register File, an 8-bit Arithmetic Logic Unit (ALU), and next-PC selection logic. Its instruction set supports LOAD, STORE, ADD, SUB, ADDI, BEQ, JUMP, and HALT operations. Functional verification was performed using nine dedicated testbenches, recording 142 checks passed and zero failures. Generic synthesis using Yosys reported 116 cells and zero structural problems. The project audit also records AMD Xilinx 7-Series technology-mapping results of 156 slice registers, 184 slice LUTs, and 170 estimated logic cells. The implementation demonstrates modular RTL design, functional verification, and synthesis analysis.

Index Terms—Harvard Architecture, Verilog HDL, single-cycle processor, instruction memory, data memory, RTL verification, Yosys.

I. INTRODUCTION

A processor executes instructions by fetching an instruction, decoding its operation, performing the required computation or memory access, and selecting the address of the next instruction. The organization of instruction and data storage is an important part of this process. Vector Cores is an educational processor project that applies Harvard Architecture, where instruction memory and data memory are separate.

The objective of this project is to implement a modular single-cycle processor in Verilog HDL and verify its functional behavior. The design uses 16-bit instructions and 8-bit data values. It supports arithmetic operations, memory access, conditional branching, jumping, and halt control. Each completed instruction is intended to execute in one clock cycle.

II. LITERATURE REVIEW

Harvard Architecture and von Neumann Architecture are two foundational computer organization models. In a von Neumann organization, instructions and data share a common memory system or path. Harvard Architecture separates instruction storage from data storage, allowing instruction fetch and data access to use distinct paths. This separation is the main architectural principle adopted in Vector Cores [2].

A basic single-cycle processor datapath commonly includes a Program Counter, instruction memory, instruction decoder, register file, arithmetic logic unit, data memory, and selection logic for write-back and next-address generation. The control unit decodes the instruction opcode and coordinates the datapath components. This modular organization follows established digital logic and processor design principles [2].

Verilog HDL provides constructs for describing digital hardware at behavioral and register-transfer levels. Simulation tools allow the designer to test module behavior and integrated execution, while synthesis tools translate RTL descriptions into a generic logic representation for structural analysis [1], [3], [4]. In this project, Icarus Verilog is used for simulation and Yosys is used for synthesis.

III. PROPOSED SOLUTION

The proposed solution is a modular 16-bit instruction, 8-bit data, single-cycle Harvard processor. The design contains a 16 × 16-bit Instruction Memory, a separate 16 × 8-bit Data Memory, a 4-bit Program Counter, a Control Unit, a 4 × 8-bit Register File, an 8-bit ALU, and next-PC selection logic. A 4-to-16 Address Decoder is instantiated internally within Data Memory.

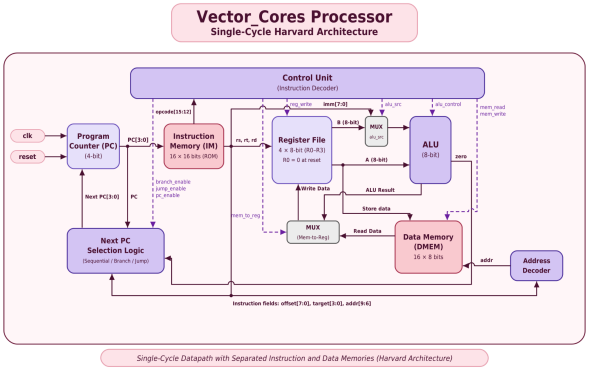


Fig. 1. Vector Cores processor block diagram supplied with the project.

A. Main Architecture Specifications

Parameter	Specification
Architecture	Harvard Architecture
Execution model	Single-cycle
Instruction / data width	16 bits / 8 bits
Program Counter	4 bits
Instruction Memory	16 × 16 bits
Data Memory	16 × 8 bits
Register File	4 × 8-bit registers (R0–R3)
Clock / reset	Rising edge / active-high synchronous

B. Hardware Modules

The Program Counter is a 4-bit synchronous register that updates on the rising clock edge. An active-high synchronous reset clears it, while its enable input controls whether it advances. The Instruction Memory stores 16 words of 16 bits and provides combinational reads. It initializes the instruction array from program.hex using $readmemh.

The Control Unit decodes the opcode and generates the signals required for register writes, memory reads and writes, ALU operation selection, operand selection, write-back selection, branching, jumping, PC enable, and HALT. The Register File contains four 8-bit registers with two combinational read ports and one synchronous write port. R0 is hardwired to zero; writes to R0 are blocked.

The ALU supports addition and subtraction and produces an 8-bit result and a zero flag. Arithmetic wraps modulo 256. Data Memory contains 16 locations of 8 bits, supports combinational reads and synchronous writes, and clears all locations synchronously during reset. Its internal 4-to-16 Address Decoder generates one-hot word selection. The Top Module connects these blocks and implements operand steering, write-back selection, branch target calculation, jump selection, and HALT behavior.

C. Instruction Set and Encoding

Instruction	Opcode	Function
LOAD	0001	R[rd] = DMEM[address]
STORE	0010	DMEM[address] = R[rs]
ADD	0011	R[rd] = R[rs] + R[rt]
SUB	0100	R[rd] = R[rs] − R[rt]
ADDI	0101	R[rd] = R[rs] + immediate
BEQ	0110	Branch if R[rs] = R[rt]
JUMP	0111	PC = target
HALT	1111	Disable PC updates; halted = 1

The opcode occupies bits [15:12]. LOAD uses [11:10] for rd and [9:6] for the data-memory address, with [5:0] set to zero. STORE uses [11:10] for rs and [9:6] for the address, with [5:0] set to zero. ADD and SUB use [11:10] for rd, [9:8] for rs, [7:6] for rt, and [5:0] for zeros. ADDI uses [11:10] for rd, [9:8] for rs, and [7:0] for the signed immediate. BEQ uses [11:10] for rs, [9:8] for rt, and [7:0] for the signed offset. JUMP uses [3:0] for the absolute target and sets [11:4] to zero. HALT sets [11:0] to zero.

For BEQ, the signed 8-bit offset is added to PC+1 and the result is truncated to the 4-bit PC address space. JUMP selects an absolute 4-bit

target. The processor supports 16 instruction addresses and 16 data-memory addresses.

IV. INDIVIDUAL CONTRIBUTIONS

The project was developed collaboratively, with primary module ownership and verification responsibilities distributed among four team members.

Team member	Contribution
Pratiti Paul	Implemented the Program Counter and Control Unit, integrated the processor, and handled system-level testing.
Anuradha Raghuwanshi	Implemented Instruction Memory, developed its testbench, and prepared program.hex.
Ganga	Implemented Data Memory and the 4-to-16 Address Decoder, along with its testbench.
Alisha Gupta	Implemented the ALU and Register File and developed their respective testbenches.

V. FUNCTIONAL VERIFICATION AND RESULTS

Nine dedicated Verilog testbenches were used to verify individual modules and integrated processor behavior. The test suite covers Program Counter operation, instruction decoding, instruction and data memory behavior, ALU operations, register-file behavior, processor execution, control flow, boundary conditions, and reset behavior.

Testbench	Passed	Failed
Program Counter	5	0
Control Unit	8	0
Instruction Memory	43	0
Data Memory	38	0
ALU	7	0
Register File	6	0
Processor Integration	7	0
Control Flow	13	0
Boundary and Reset	15	0
Total	142	0

The control-flow tests include BEQ taken and not-taken cases, forward and backward branches, JUMP, and HALT. Boundary and reset tests include negative branch offsets, PC wraparound, and reset behavior across processor components. The recorded regression result is 142 checks passed and zero failed.

A. Processor-Level Program

The program in program.hex initializes R1 to 5 and R2 to 3, adds the registers, stores the result at Data Memory location 2, subtracts R2 from R1, loads the stored value into R3, increments R3, and halts.

Address	Hex	Instruction	Result
0x0	5405	ADDI R1, R0, 5	R1 = 5
0x1	5803	ADDI R2, R0, 3	R2 = 3
0x2	3580	ADD R1, R1, R2	R1 = 8
0x3	2480	STORE R1, [2]	DMEM[2] = 8
0x4	4580	SUB R1, R1, R2	R1 = 5
0x5	1C80	LOAD R3, [2]	R3 = 8
0x6	5F01	ADDI R3, R3, 1	R3 = 9
0x7	F000	HALT	PC = 7

The expected final architectural state is R0=0, R1=5, R2=3, R3=9, DMEM[2]=8, PC=7, and halted=1. The processor-level simulation verifies these values and confirms that the PC remains at 7 after HALT. Instruction-memory locations 0x8 through 0xF contain F000 (HALT).

VI. SYNTHESIS AND RESOURCE ANALYSIS

The complete processor hierarchy was synthesized using Yosys 0.70 with top_module selected as the top-level module. The generic synthesis flow included hierarchy checking, process lowering, optimization, memory processing, structural checking, and statistics reporting.

Generic cell type	Count
$mux	47
$eq	18
$sdffe	17
$and	9
$logic_not	5
$reduce_bool	5
$dffe	4
$add	3
$not	2
$pmux	2
$shl	1
$sub	1
$logic_and	1
$logic_or	1
Total	116

The generic structural check reported zero problems. Yosys also reported that the Data Memory array was replaced with a list of registers. This is consistent with the RTL behavior that synchronously clears all 16 memory locations during reset.

A. Xilinx 7-Series Technology Mapping

The project audit records the following technology-mapping results for the AMD Xilinx 7-Series family (xc7):

Resource	Count
Slice registers (FDRE)	156
Slice LUTs	184
Carry chains (CARRY4)	6
Clock buffers (BUFG)	1
Input buffers (IBUF)	2
Output buffers (OBUF)	38
Estimated logic cells	170

The recorded register breakdown is 128 Data Memory bits, 24 writable Register File bits (R1–R3), and 4 Program Counter bits. The LUT breakdown is LUT2: 14, LUT3: 28, LUT4: 32, LUT5: 85, and LUT6: 25. These are technology-mapping results recorded by the project audit; they are not measured board utilization or timing results. The audit did not provide the exact synth_xilinx command or a separate target-mapping log path.

The generic synthesis statistics are also not timing or power measurements. Maximum operating frequency, critical-path delay, and power consumption were not measured.

VII. LIMITATIONS AND FUTURE IMPROVEMENTS

The current implementation is a simplified educational processor. Data Memory is mapped to registers in the reported synthesis flow because its synchronous reset clears all 16 locations in one clock cycle. Instruction Memory uses asynchronous combinational reads, which may need a different implementation for FPGA block RAM configurations. The 4-bit address buses limit both instruction and data memories to 16 locations. Future improvements may include additional arithmetic and logical instructions, wider datapaths, larger memories and register files, FPGA-oriented memory inference, board-level testing, timing analysis, and power measurement.

VIII. PERFORMANCE METRICS

The single-cycle execution model has a CPI of 1 for each completed instruction and an ideal throughput of one instruction per clock cycle. This is an architectural property of the design. Actual operating frequency and instructions per second have not been measured.

IX. CONCLUSION

Vector Cores demonstrates the design and implementation of a modular single-cycle processor using Harvard Architecture and Verilog HDL. The integrated design supports instruction fetching, decoding, register operations, arithmetic processing, data-memory access, and control-flow management. Nine testbenches completed 142 checks with zero failures. Generic Yosys synthesis reported 116 cells and zero structural problems, while the project audit also recorded Xilinx 7-Series technology-mapping results. FPGA timing and board-level measurements remain future work.

REFERENCES

[1] IEEE, “IEEE Standard for Verilog Hardware Description Language,” IEEE Std 1364-2001.

[2] C. H. Roth, Jr. and L. L. Kinney, Fundamentals of Logic Design, 7th ed. Boston, MA, USA: Cengage Learning, 2014.

[3] YosysHQ, “Yosys Open SYnthesis Suite Documentation.” [Online]. Available: https://yosyshq.readthedocs.io/

[4] Icarus Verilog Project, “Icarus Verilog Documentation.” [Online]. Available: https://steveicarus.github.io/iverilog/